

Guía del alumno

Módulo 6, Visualización de datos

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después de la clase. Trae el mismo código y las mismas figuras que viste en el laboratorio, con la explicación escrita y la solución de cada ejercicio.

Este módulo se dicta en dos clases. Los bloques 1 a 4 son de la primera y los bloques 5 a 7 de la segunda. Cuando vuelvas a abrir el notebook, ejecútalo desde arriba.

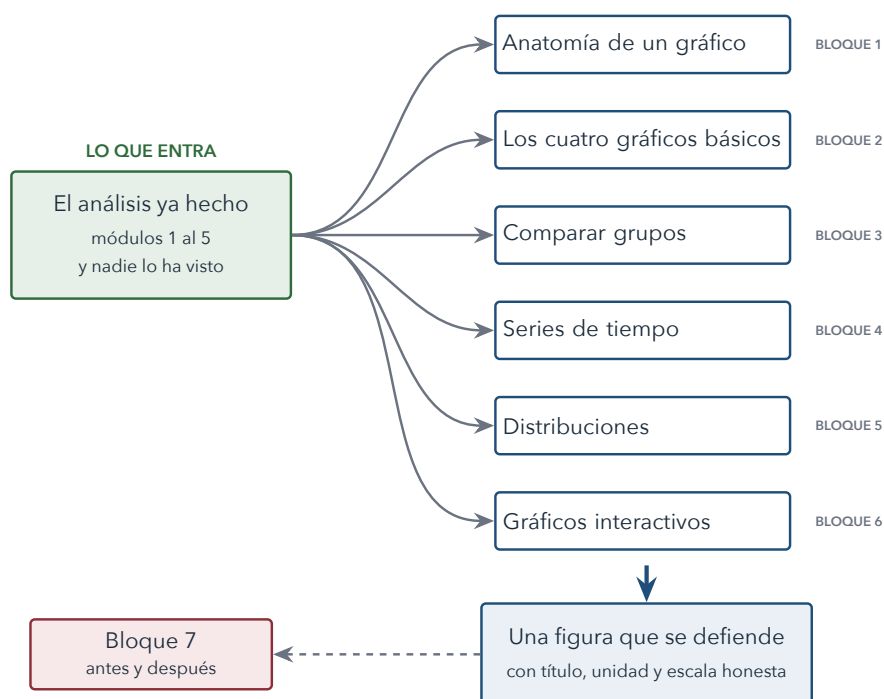
Lo que vas a poder hacer al terminar

- Armar un gráfico controlando cada parte, y saber qué es el mínimo que lleva.
- Elegir el tipo de gráfico a partir de la pregunta, no del gusto.
- Comparar siete grupos en una figura, con la escala resuelta.
- Resumir una serie de 8.784 puntos sin perder la historia.
- Publicar un gráfico interactivo en un archivo que se manda por correo.
- Revisar un gráfico propio con una lista de cinco cosas, antes de mandarlo.

Todas las figuras de esta guía salen de la corrida verificada del notebook, incluidas las de Plotly. Son exactamente los gráficos que ves en pantalla. Los datos son sintéticos, y donde una cifra del laboratorio se aleja de la realidad del sistema chileno, la guía lo dice.

1. El módulo en una imagen

Entran cinco módulos de análisis que nadie ha visto. Sale una figura que se defiende sola.



La imagen del informe que nadie leyó

Hay un informe, en alguna carpeta, que tomó tres meses. Tiene los datos bien buscados, la limpieza bien hecha, los cruces correctos y las conclusiones sólidas. Nadie lo leyó.

Y hay otro, de dos páginas, que alguien armó en una tarde y que se citó en tres reuniones. La diferencia no fue el rigor. Fue que el segundo tenía un gráfico que se entendía en diez segundos y el primero tenía una tabla de cuarenta filas.

El que decide tiene quince minutos y ocho temas. Si tu trabajo necesita que alguien se siente contigo a que se lo expliques, tu trabajo no está terminado.

El gráfico no es la decoración del análisis. Es la parte del análisis que viaja.

Lo que este módulo no es

No es un curso de diseño. No hay tipografía, ni composición, ni paletas de moda. Hay tres preguntas prácticas, qué lleva un gráfico para que se entienda solo, cuál corresponde a cada pregunta, y cómo se revisa el propio antes de mandarlo.

Y no trae operaciones nuevas. Todo lo que se dibuja acá sale de cosas que sabes hacer desde el Módulo 2.

2. Bloque 1, anatomía de un gráfico

Qué se vio

Hacer un gráfico cuesta una línea, y ese es el problema. El que sale en una línea sirve para mirarlo uno mismo mientras trabaja, y no sirve para nada más.

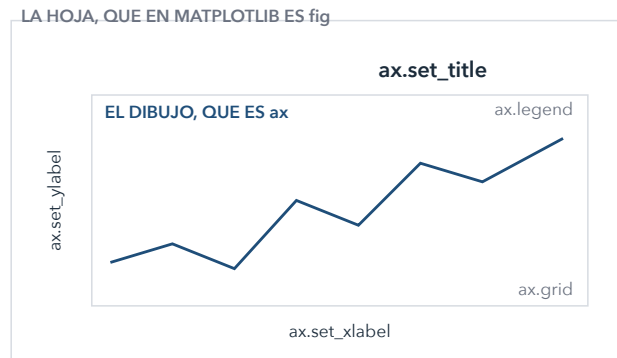


fig es la hoja y ax es el dibujo. En una hoja puede haber varios dibujos, y por eso `plt.subplots(2, 2)` devuelve una hoja y cuatro dibujos. Casi todo lo que uno quiere cambiar, el título, los rótulos, la escala, la grilla y la leyenda, se le pide al dibujo, no a la hoja. Lo que se le pide a la hoja es el título general y el ajuste del espacio entre dibujos.

El código, paso a paso

1.1 Los datos del módulo

```
import pandas as pd
import matplotlib.pyplot as plt

gen = pd.read_csv("generacion.csv", parse_dates=["fecha"])
centrales = pd.read_csv("centrales.csv")
demanda = pd.read_csv("demanda.csv", parse_dates=["fecha"])
gen = gen.merge(centrales, on="central")

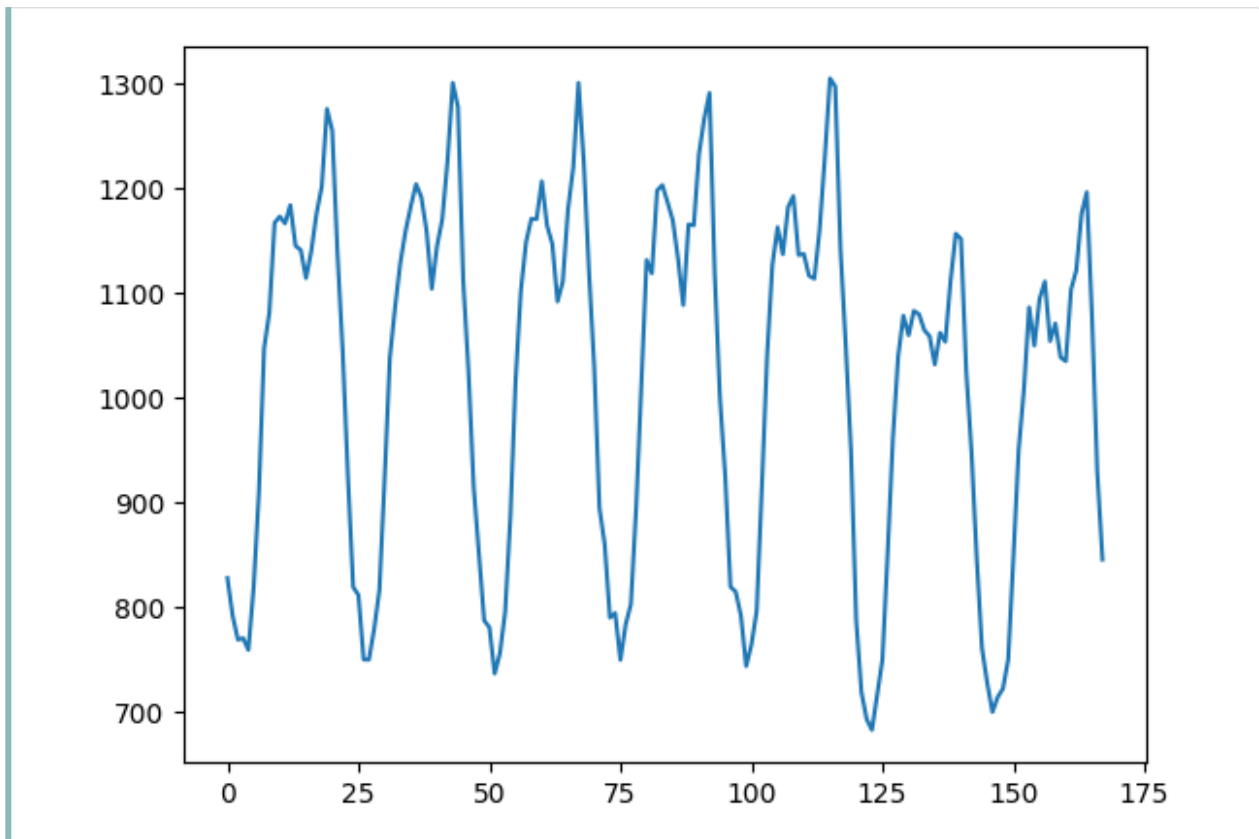
print(gen.shape, demanda.shape)
```

```
(175680, 8) (61488, 4)
```

Las mismas tres tablas de siempre. Este módulo no trae operaciones nuevas, trae decisiones nuevas.

1.2 El gráfico de una línea

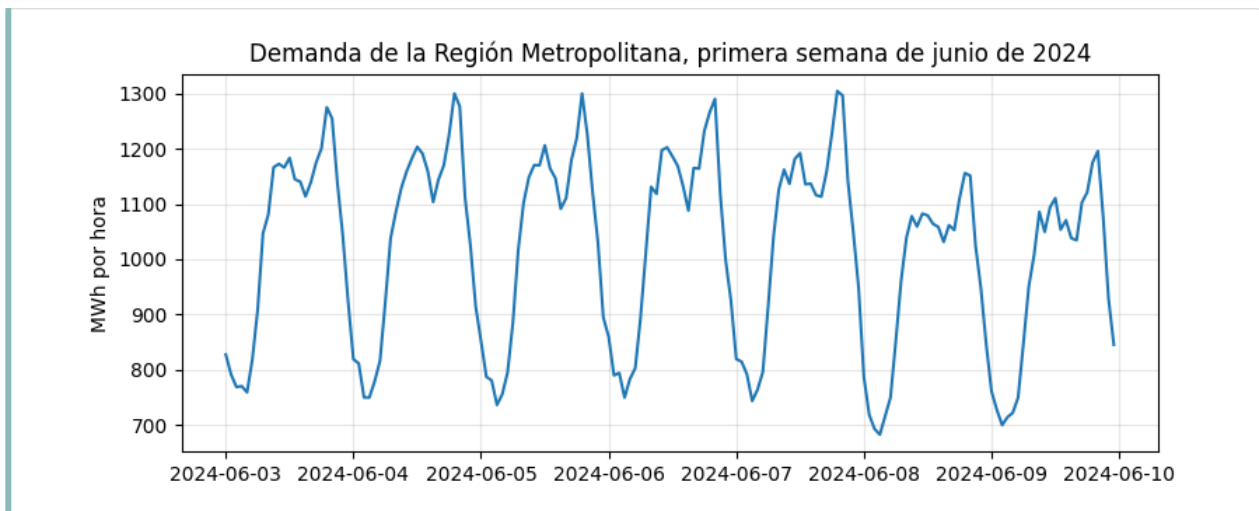
```
# El gráfico más corto posible. Sale, y no dice nada.  
met = demanda[demanda["region"] == "Metropolitana"]  
semana = met[met["fecha"].between("2024-06-03", "2024-06-09")]  
  
fig, ax = plt.subplots()  
ax.plot(semana["mwh"].values)  
plt.show()
```



Sale, se ve, y no dice de qué región es, ni qué unidad tiene el eje vertical, ni qué son los números del eje horizontal, que son las horas de la semana.

1.3 El mismo dato, con lo mínimo

```
# El mismo dato, con lo mínimo que hace falta para entenderlo.
fig, ax = plt.subplots(figsize=(9, 3.5))
ax.plot(semana["fecha"] + pd.to_timedelta(semana["hora"], unit="h"), semana["mwh"])
ax.set_title("Demanda de la Región Metropolitana, primera semana de junio de 2024")
ax.set_xlabel("")
ax.set_ylabel("MWh por hora")
ax.grid(alpha=0.3)
plt.show()
```



Cuatro líneas más. Fíjate en el detalle de sumar la hora a la fecha con `pd.to_timedelta`. Sin eso, las veinticuatro horas de un día caen en el mismo punto del eje, que es un error muy común con datos horarios.

1.4 La hoja y el dibujo

```
# fig y ax son dos cosas distintas y conviene tenerlo claro.
# fig es la hoja, ax es el dibujo. En una hoja puede haber varios dibujos.
fig, ejes = plt.subplots(1, 2, figsize=(10, 3))
ejes[0].plot(semana["mwh"].values)
ejes[0].set_title("un dibujo")
ejes[1].bar(["a", "b", "c"], [3, 1, 2])
ejes[1].set_title("y otro en la misma hoja")
fig.suptitle("Una hoja, dos dibujos", y=1.04)
plt.show()
```

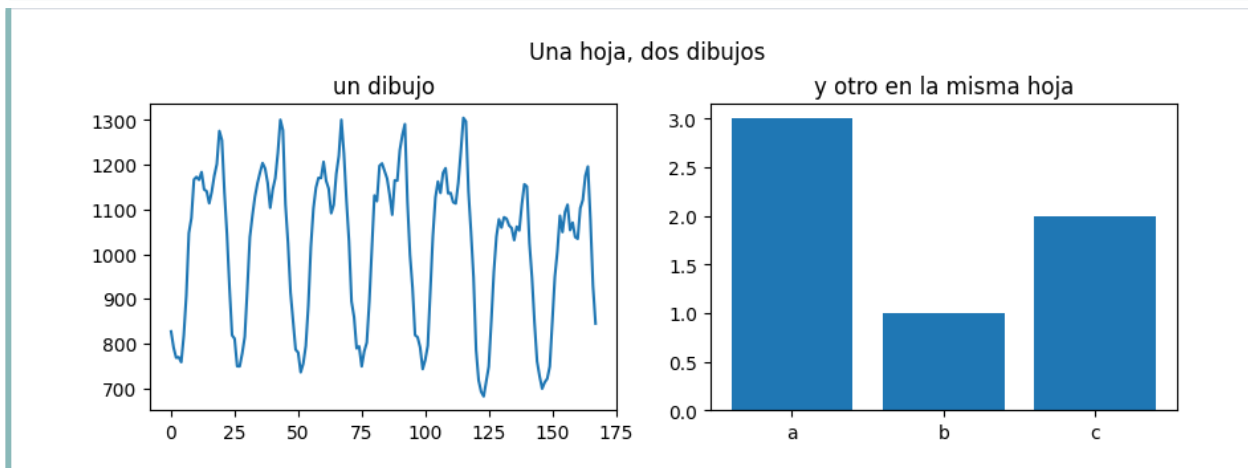


fig es la hoja y ax es el dibujo. En una hoja puede haber varios dibujos, y casi todo lo que quieres cambiar se le pide al dibujo. Esta distinción es la llave del módulo.

La solución del ejercicio

El ejercicio pedía cambiar la región y las fechas.

```
region, desde, hasta = "Valparaíso", "2024-01-08", "2024-01-14"
trozo = demanda[(demanda["region"] == region) & (demanda["fecha"].between(desde, hasta))]

fig, ax = plt.subplots(figsize=(9, 3))
ax.plot(trozo["fecha"] + pd.to_timedelta(trozo["hora"], unit="h"), trozo["mwh"])
ax.set_title(f'Demanda de {region}, del {desde} al {hasta}')
ax.set_ylabel("MWh por hora")
plt.show()
```

El título se arma con una f-string, así que cambia solo. Ese hábito evita el error más común de todos, mandar un gráfico con el título del gráfico anterior.

LO MÍNIMO QUE LLEVA UN GRÁFICO

Un **título** que diga qué se está mirando, con el periodo adentro. Si el título no alcanza para entender el gráfico, el título está mal.

Los **ejes rotulados con su unidad**. MWh por hora, no solo MWh. El eje que se explica solo, como una fecha, no necesita rótulo.

Una **escala honesta**. En un gráfico de barras el eje parte en cero, y si no, hay que poder justificarlo. En uno de líneas el eje se ajusta al rango de los datos, porque lo que se lee es la forma y no el largo.

Y nada más. Cada elemento que agregas tiene que ganarse el lugar. Un gráfico mejora casi siempre sacando cosas, no agregándolas.

Referencia rápida

Instrucción	Para qué sirve
<code>fig, ax = plt.subplots(figsize=)</code>	La hoja y el dibujo
<code>plt.subplots(2, 2)</code>	Una hoja con cuatro dibujos
<code>ax.set_title(t)</code>	El título del dibujo
<code>ax.set_xlabel · ax.set_ylabel</code>	Los rótulos, con la unidad
<code>ax.legend()</code>	La leyenda, si hay varias series
<code>ax.grid(alpha=0.3)</code>	Una grilla suave
<code>fig.suptitle(t)</code>	El título de toda la hoja
<code>fig.tight_layout()</code>	Acomoda el espacio entre dibujos
<code>fig.savefig(f, dpi=150, bbox_inches="tight")</code>	Guardar en archivo, sin borde sobrante

Errores comunes

Lo que aparece en pantalla	Qué pasó
Todas las horas del día en el mismo punto	Falta sumar la hora a la fecha con <code>to_timedelta</code> .
Los títulos se pisan con los ejes de abajo	Falta <code>fig.tight_layout()</code> .
El archivo guardado tiene un borde blanco enorme	Falta <code>bbox_inches="tight"</code> en el <code>savefig</code> .
El gráfico sale dos veces en Colab	Sobra el <code>plt.show()</code> o falta el punto y coma, según el caso.

Tres preguntas de repaso

1. Cuál es el mínimo que lleva un gráfico

Título con el periodo, ejes rotulados con su unidad, escala honesta, y nada más.

2. Qué diferencia hay entre `fig` y `ax`

`fig` es la hoja y `ax` es el dibujo. En una hoja puede haber varios dibujos, y casi todo lo que uno cambia se le pide al dibujo.

3. Por qué conviene armar el título con una f-string

Porque así cambia solo cuando cambian los datos, y no se manda un gráfico con el título del anterior.

3. Bloque 2, los cuatro gráficos básicos

Qué se vio

El tipo de gráfico sale de la pregunta, no del gusto, y casi todas las preguntas caben en cuatro.

LA PREGUNTA DECIDE EL GRÁFICO, NO EL GUSTO

LÍNEA

Cómo cambió esto en el tiempo
Eje horizontal con orden. Casi siempre tiempo.

BARRAS

Cuál categoría es mayor
Horizontales si los nombres son largos. Siempre ordenadas.

DISPERSIÓN

Estos dos números se mueven juntos
Un punto por unidad. El color agrega una tercera variable.

HISTOGRAMA

Cómo se reparte esta variable
El número de cajones cambia la historia. Pruébalo.

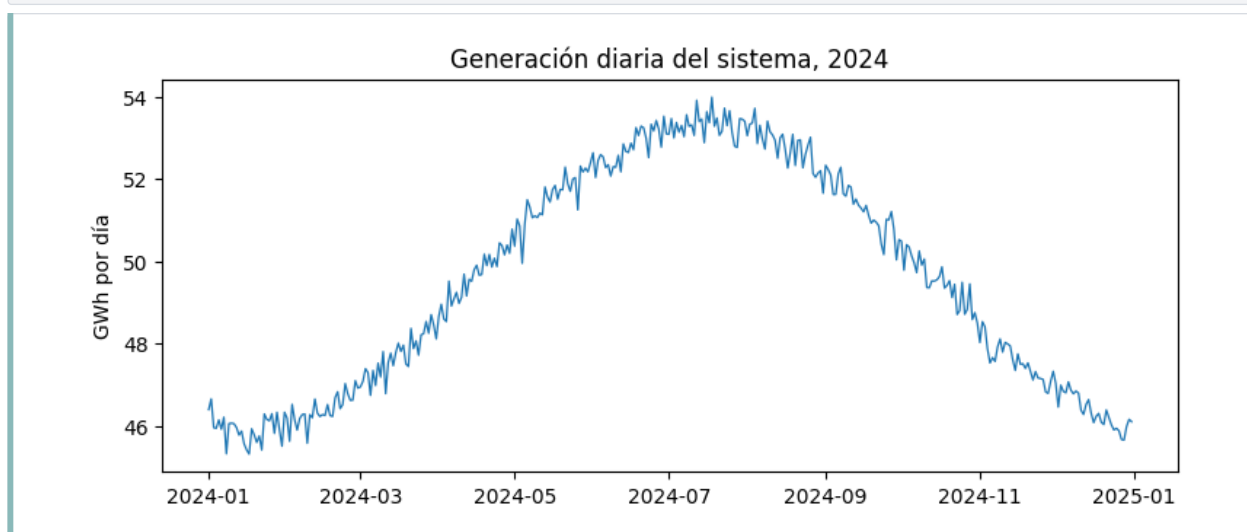
Casi todo lo que se publica en el sector cabe en estos cuatro. El de torta no está, y es a propósito. El ojo compara largos mucho mejor que ángulos, así que un gráfico de barras ordenado hace el mismo trabajo y se lee más rápido.

El código, paso a paso

2.1 Línea

```
# Línea, cuando el eje horizontal tiene orden. Casi siempre, tiempo.
diaria = gen.groupby("fecha")["mwh"].sum() / 1000

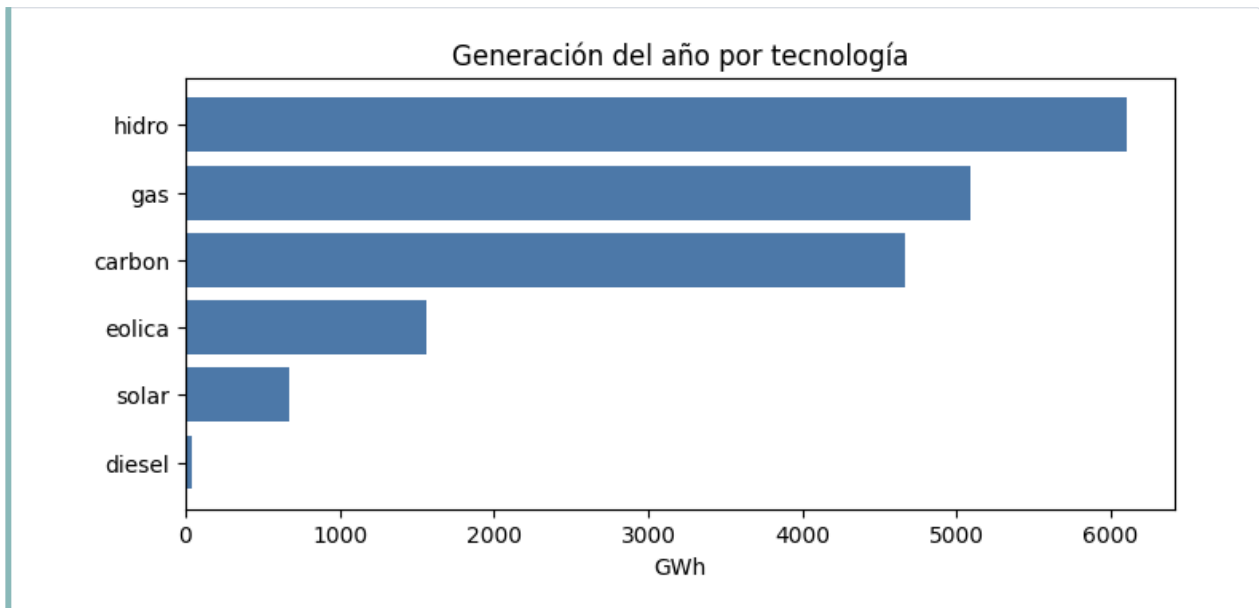
fig, ax = plt.subplots(figsize=(9, 3.5))
ax.plot(diaria.index, diaria.values, linewidth=0.9)
ax.set_title("Generación diaria del sistema, 2024")
ax.set_ylabel("GWh por día")
plt.show()
```



Línea porque el eje horizontal tiene orden. Si fueran categorías sin orden, la línea sugeriría una continuidad que no existe.

2.2 Barras

```
# Barras, cuando se comparan categorías. Horizontales si los nombres son largos.  
por_tec = gen.groupby("tecnologia")["mwh"].sum().sort_values() / 1000  
  
fig, ax = plt.subplots(figsize=(8, 3.5))  
ax.barh(por_tec.index, por_tec.values, color="#4C78A8")  
ax.set_title("Generación del año por tecnología")  
ax.set_xlabel("GWh")  
plt.show()
```

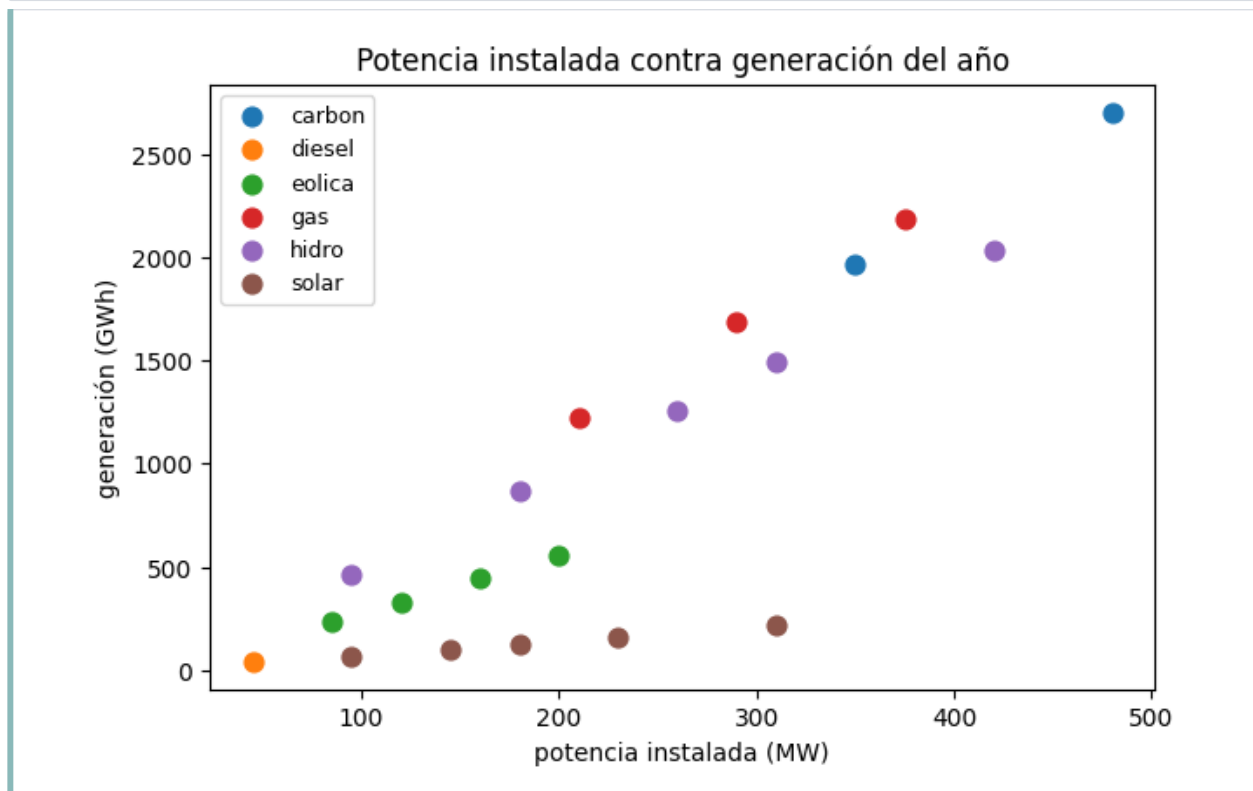


Barras porque comparamos categorías. Horizontales porque los nombres son largos, y ordenadas porque sin orden el lector tiene que buscar. El `sort_values` antes de dibujar es lo que hace la diferencia.

2.3 Dispersión

```
# Dispersión, cuando la pregunta es si dos números se mueven juntos.
resumen = gen.groupby(["central", "tecnologia", "potencia_mw"])["mwh"].sum().reset_index()
resumen["gwh"] = resumen["mwh"] / 1000

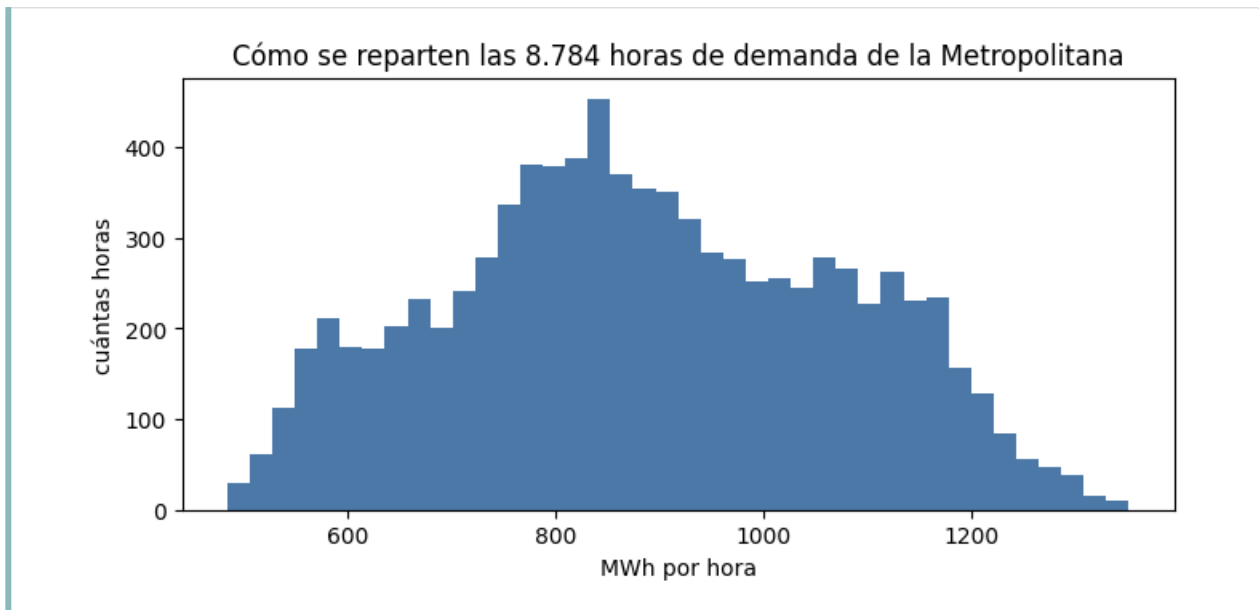
fig, ax = plt.subplots(figsize=(7, 4.5))
for tec, grupo in resumen.groupby("tecnologia"):
    ax.scatter(grupo["potencia_mw"], grupo["gwh"], label=tec, s=60)
ax.set_title("Potencia instalada contra generación del año")
ax.set_xlabel("potencia instalada (MW)")
ax.set_ylabel("generación (GWh)")
ax.legend(fontsize=9)
plt.show()
```



Dispersión porque la pregunta es si dos números se mueven juntos. El color agrega la tecnología sin costo. La relación existe y no es una recta, porque la tecnología manda.

2.4 Histograma

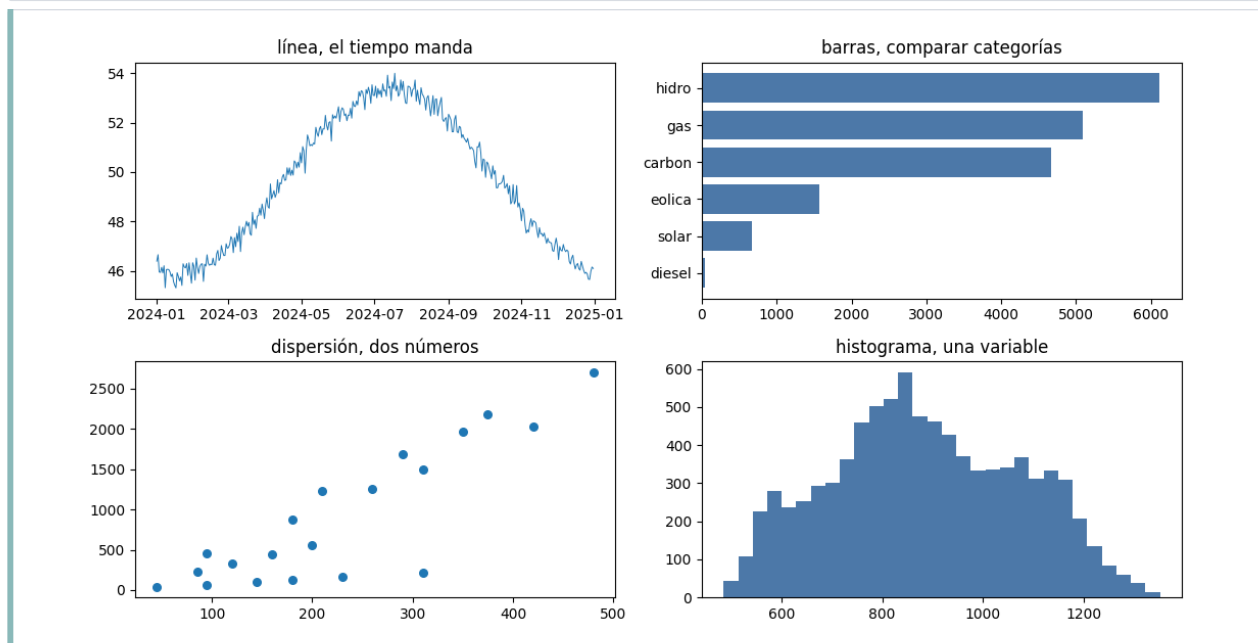
```
# Histograma, cuando la pregunta es cómo se reparte una sola variable.  
fig, ax = plt.subplots(figsize=(8, 3.5))  
ax.hist(met["mwh"], bins=40, color="#4C78A8")  
ax.set_title("Cómo se reparten las 8.784 horas de demanda de la Metropolitana")  
ax.set_xlabel("MWh por hora")  
ax.set_ylabel("cuántas horas")  
plt.show()
```



Histograma porque la pregunta es cómo se reparte una sola variable. El número de cajones cambia la forma, así que conviene probar dos o tres.

2.5 Los cuatro juntos

```
# Los cuatro juntos, para tenerlos a la vista.
fig, ejes = plt.subplots(2, 2, figsize=(11, 6))
ejes[0, 0].plot(diaria.index, diaria.values, linewidth=0.7)
ejes[0, 0].set_title("línea, el tiempo manda")
ejes[0, 1].barh(por_tec.index, por_tec.values, color="#4C78A8")
ejes[0, 1].set_title("barras, comparar categorías")
ejes[1, 0].scatter(resumen["potencia_mw"], resumen["gwh"], s=30)
ejes[1, 0].set_title("dispersión, dos números")
ejes[1, 1].hist(met["mwh"], bins=30, color="#4C78A8")
ejes[1, 1].set_title("histograma, una variable")
fig.tight_layout()
plt.show()
```



La lámina de referencia. Fíjate en el `tight_layout` del final, sin él los títulos se pisan con los ejes de abajo.

La solución del ejercicio

El ejercicio pedía las barras de la demanda por región.

```
por_region = demanda.groupby("region")["mwh"].sum().sort_values() / 1000

fig, ax = plt.subplots(figsize=(8, 3))
ax.barh(por_region.index, por_region.values, color="#72B7B2")
ax.set_title("Demanda total del año por región, 2024")
ax.set_xlabel("GWh")
plt.show()
```

La Metropolitana queda muy por encima de todas, así que las otras seis casi no se distinguen entre sí. Ese problema se resuelve en el bloque 3.

Referencia rápida

Instrucción	Para qué sirve
<code>ax.plot(x, y)</code>	Línea, cuando el eje horizontal tiene orden
<code>ax.bar</code> · <code>ax.barh</code>	Barras verticales y horizontales
<code>ax.scatter(x, y, s=, label=)</code>	Dispersión, con tamaño y leyenda
<code>ax.hist(serie, bins=)</code>	Histograma, una variable
<code>serie.sort_values()</code>	Ordenar antes de dibujar barras
<code>ax.bar_label(ax.containers[0])</code>	El valor al lado de cada barra

Tres preguntas de repaso

1. Cómo se elige el tipo de gráfico

Por la pregunta. Cómo cambió en el tiempo, línea. Cuál es mayor, barras. Se mueven juntos, dispersión. Cómo se reparte, histograma.

2. Por qué no hay gráfico de torta

Porque el ojo compara largos mucho mejor que ángulos. Una barra ordenada da la misma información y se lee más rápido, y además aguanta más categorías.

3. Qué diferencia hay entre un histograma y un gráfico de barras

En el histograma el eje horizontal es numérico y el ancho de la barra significa el rango del cajón. En el de barras son categorías y el ancho no significa nada.

4. Bloque 3, comparar grupos

Qué se vio

Comparar de verdad es poner todo en el mismo gráfico con la misma escala. Y cuando un caso aplasta a los demás, hay dos salidas honestas, sacarlo de la escala y decirlo, o normalizar.

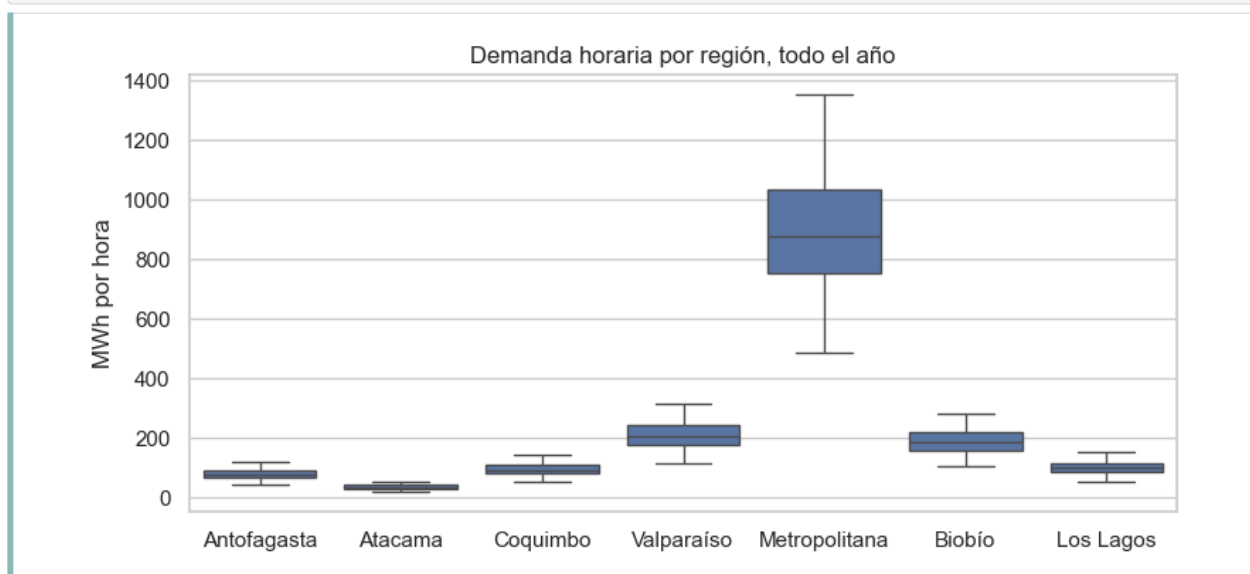
El código, paso a paso

3.1 El boxplot

```
import seaborn as sns

sns.set_theme(style="whitegrid")

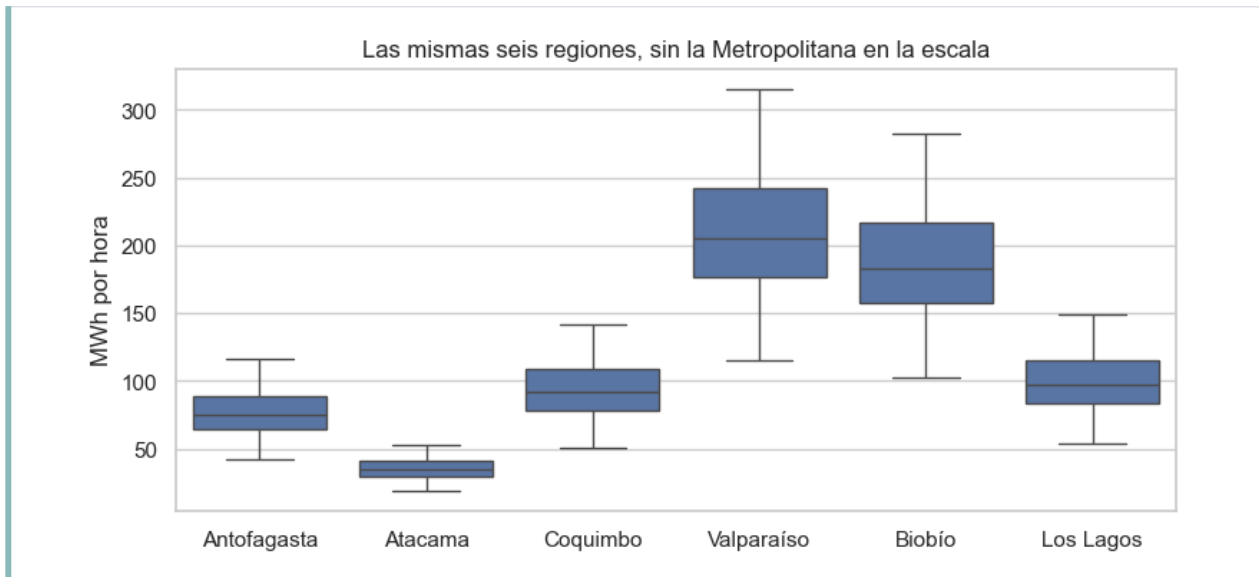
# Un boxplot compara la distribución de varios grupos de una vez.
fig, ax = plt.subplots(figsize=(9, 4))
sns.boxplot(data=demanda, x="region", y="mwh", ax=ax)
ax.set_title("Demanda horaria por región, todo el año")
ax.set_ylabel("MWh por hora")
ax.set_xlabel("")
plt.show()
```



La línea del medio es la mediana, la caja va del 25 al 75 por ciento y los bigotes llegan hasta 1,5 veces el largo de la caja. Lo de más allá son puntos sueltos, que no son errores sino datos. La Metropolitana ocupa toda la altura y las otras seis quedan aplastadas.

3.2 Sin la Metropolitana

```
# La Metropolitana aplasta a las demás. Sin ella se ven las otras seis.  
sin_rm = demanda[demanda["region"] != "Metropolitana"]  
  
fig, ax = plt.subplots(figsize=(9, 4))  
sns.boxplot(data=sin_rm, x="region", y="mwh", ax=ax)  
ax.set_title("Las mismas seis regiones, sin la Metropolitana en la escala")  
ax.set_ylabel("MWh por hora")  
ax.set_xlabel("")  
plt.show()
```



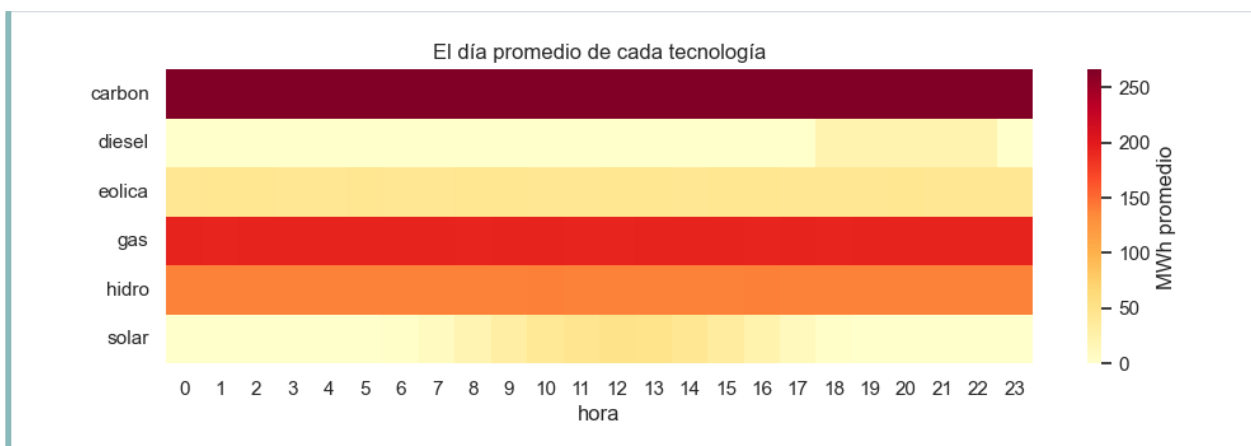
La misma tabla, sacando un caso de la escala. Es legítimo y es lo que uno hace en la práctica, con una condición, que el título lo diga.

3.3 El mapa de calor

```
# Un mapa de calor muestra dos categorías y un número, todo de una vez.
perfil = gen.pivot_table(index="tecnologia", columns="hora", values="mwh", aggfunc="mean")
# El promedio de cada fila, que es lo que da el color. Es por central, no el total.
print(perfil.mean(axis=1).round(1).sort_values(ascending=False).to_string())

fig, ax = plt.subplots(figsize=(11, 3))
sns.heatmap(perfil, cmap="YlOrRd", ax=ax, cbar_kws={"label": "MWh promedio"})
ax.set_title("El día promedio de cada tecnología")
ax.set_xlabel("hora")
ax.set_ylabel("")
plt.show()
```

```
tecnologia
carbon    265.6
gas       193.3
hidro     139.0
eolica     44.6
solar      15.3
diesel      4.6
```

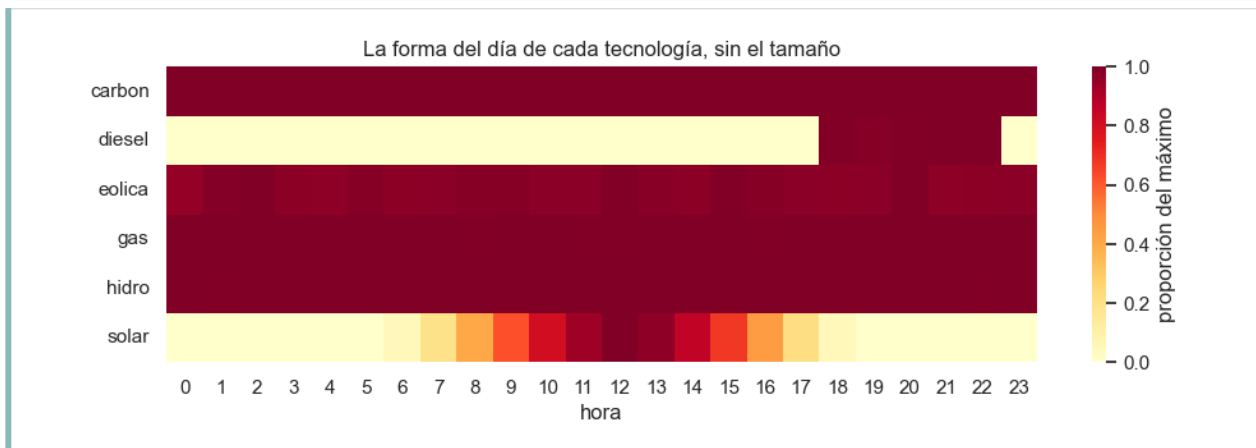


Dos categorías y un número, todo en una figura. La fila más caliente es la del carbón, con 265,6 MWh, después el gas con 193,3 y la hidro con 139,0. El mapa promedia por central, y el carbón tiene centrales grandes que generan parejo. La hidro es la que más genera en total, pero lo reparte entre cinco centrales. Más allá de esas tres filas no se ve mucho más.

3.4 El mismo mapa, normalizado

```
# El mismo mapa, con cada fila normalizada por su propio máximo.
# Ahora se compara la forma del día, no el tamaño de la tecnología.
perfil_n = perfil.div(perfil.max(axis=1), axis=0)

fig, ax = plt.subplots(figsize=(11, 3))
sns.heatmap(perfil_n, cmap="YlOrRd", ax=ax, cbar_kws={"label": "proporción del máximo"})
ax.set_title("La forma del día de cada tecnología, sin el tamaño")
ax.set_xlabel("hora")
ax.set_ylabel("")
plt.show()
```



Una línea más y aparece la información. La solar es una banda de mediodía, el diésel una banda de noche y la hidro una franja pareja.

La solución del ejercicio

El ejercicio pedía el mapa de la demanda por región.

```
perfil_d = demanda.pivot_table(index="region", columns="hora", values="mwh", aggfunc="mean")

fig, ax = plt.subplots(figsize=(11, 3))
sns.heatmap(perfil_d.div(perfil_d.max(axis=1), axis=0), cmap="YlOrRd", ax=ax)
ax.set_title("La forma del día de cada región, sin el tamaño")
plt.show()
```

Sale casi plano, porque las siete regiones del curso tienen el mismo perfil escalado por población. Es el artefacto de los datos sintéticos que ya se advirtió en el Módulo 2, y este gráfico lo muestra mejor que cualquier tabla.

DOS MAPAS DE CALOR, DOS PREGUNTAS

El primero responde cuánto genera a cada hora una central promedio de cada tecnología. Por eso la fila más caliente es la del carbón, que tiene centrales grandes y parejas, aunque la que más genera en total sea la hidro, que lo reparte entre cinco centrales.

El segundo responde a qué hora genera cada tecnología. Todas las filas usan el mismo rango, así que se ve la forma.

Es el mismo dato y la misma función. Lo que cambió es una línea de normalización, igual que en el Módulo 5.

Referencia rápida

Instrucción	Para qué sirve
<code>sns.set_theme(style="whitegrid")</code>	Un estilo base decente para todo el notebook
<code>sns.boxplot(data=, x=, y=, ax=)</code>	Comparar la distribución de varios grupos
<code>sns.violinplot(..., inner="quartile")</code>	Lo mismo, mostrando la forma completa
<code>sns.heatmap(tabla, cmap=, ax=)</code>	Dos categorías y un número
<code>cbar_kws={"label": t}</code>	Rotular la barra de color
<code>tabla.div(tabla.max(axis=1), axis=0)</code>	Normalizar cada fila antes del mapa

Tres preguntas de repaso

1. Qué hacer cuando un grupo aplasta a los demás en la escala

Sacarlo de la escala y decirlo en el título, o normalizar. Lo que no se puede es ponerle otra escala a uno solo dentro del mismo gráfico.

2. Qué muestra un mapa de calor que una tabla no

La forma, de un vistazo. Dos categorías y un número al mismo tiempo, sin que el lector tenga que comparar cifras.

3. Qué cambia normalizar cada fila del mapa

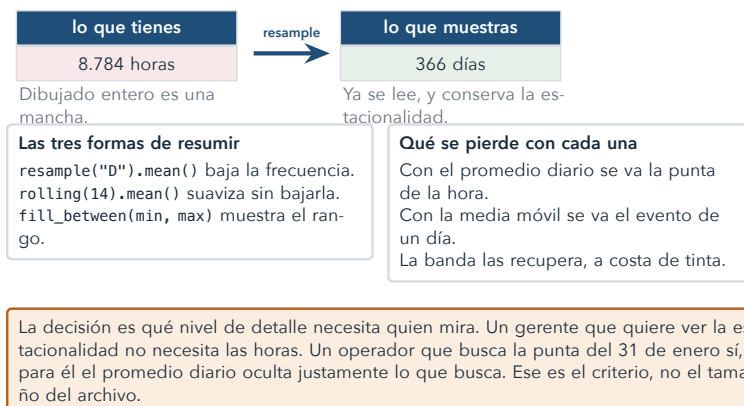
La pregunta que responde. Pasa de cuánto genera cada uno a a qué hora genera cada uno.

5. Bloque 4, series de tiempo

Qué se vio

Una serie horaria de un año son 8.784 puntos. Dibujados todos, es una mancha. Entregarlos es entregarle al lector el trabajo de resumir, que era el tuyo.

8.784 PUNTOS NO SE DIBUJAN, SE RESUMEN

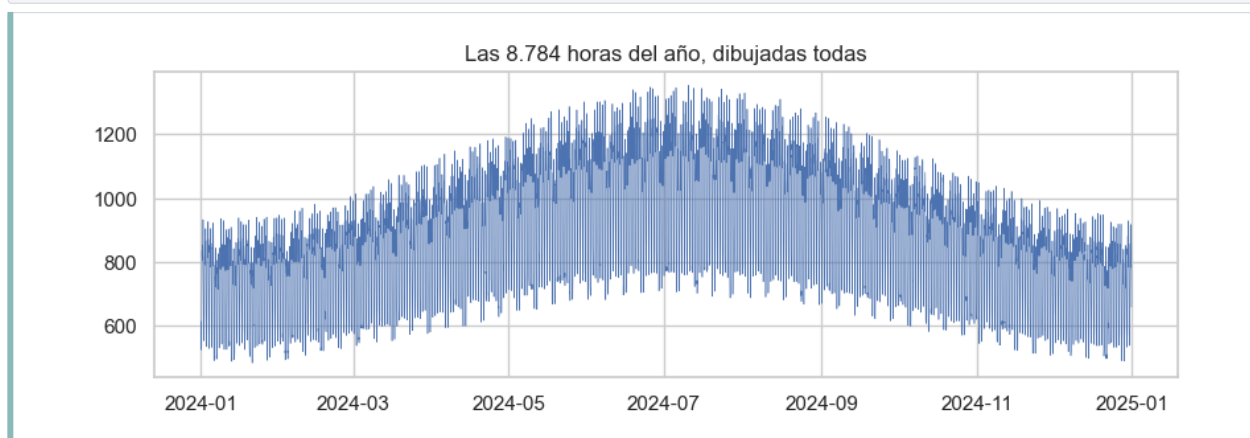


El código, paso a paso

4.1 Todos los puntos

```
# 8.784 puntos en una línea es un borrón. Esto es lo que NO hay que entregar.
met_h = met.copy()
met_h["instante"] = met_h["fecha"] + pd.to_timedelta(met_h["hora"], unit="h")
met_h = met_h.set_index("instante").sort_index()

fig, ax = plt.subplots(figsize=(10, 3))
ax.plot(met_h.index, met_h["mwh"], linewidth=0.4)
ax.set_title("Las 8.784 horas del año, dibujadas todas")
plt.show()
```

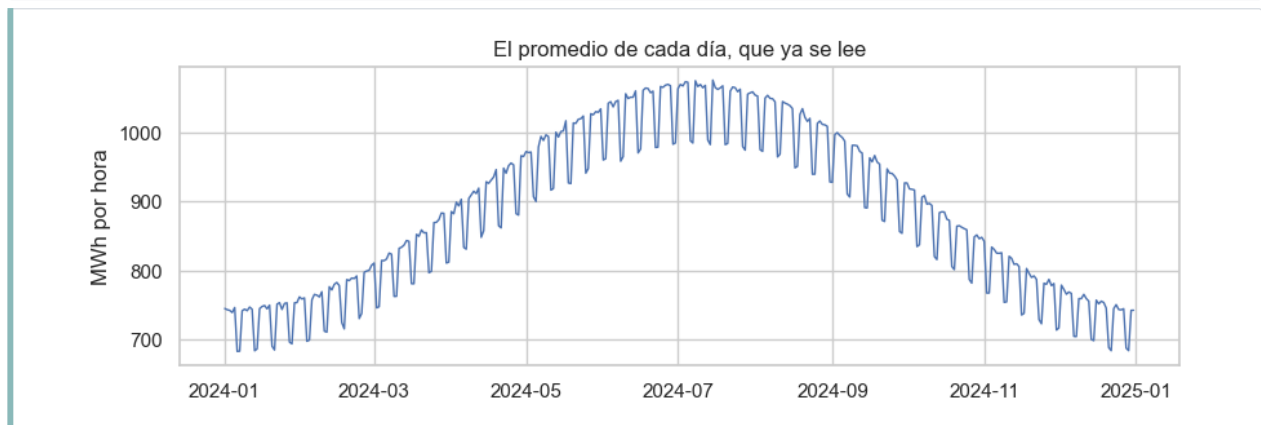


Esto es lo que no hay que entregar. Se puede concluir que sube en invierno, y nada más.

4.2 El promedio de cada día

```
# resample agrupa por tiempo. D es día, W semana, ME fin de mes.
por_dia = met_h["mwh"].resample("D").mean()

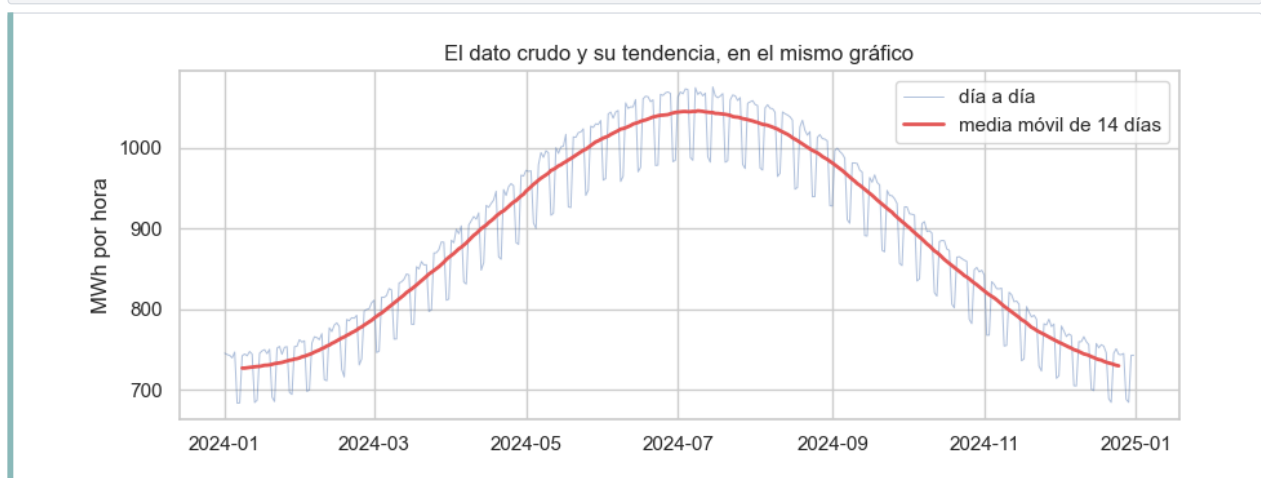
fig, ax = plt.subplots(figsize=(10, 3))
ax.plot(por_dia.index, por_dia.values, linewidth=1)
ax.set_title("El promedio de cada día, que ya se lee")
ax.set_ylabel("MWh por hora")
plt.show()
```



resample es agrupar por tiempo, la misma operación del Módulo 2 con la clave en el índice. D es día, W es semana y ME es fin de mes. Ya se lee la estacionalidad, y se perdió la punta de cada hora.

4.3 La media móvil

```
# Una media móvil suaviza sin cambiar la escala del eje.
fig, ax = plt.subplots(figsize=(10, 3.5))
ax.plot(por_dia.index, por_dia.values, linewidth=0.7, alpha=0.45, label="día a día")
ax.plot(por_dia.index, por_dia.rolling(14, center=True).mean(),
        linewidth=2, color="#E45756", label="media móvil de 14 días")
ax.set_title("El dato crudo y su tendencia, en el mismo gráfico")
ax.set_ylabel("MWh por hora")
ax.legend()
plt.show()
```

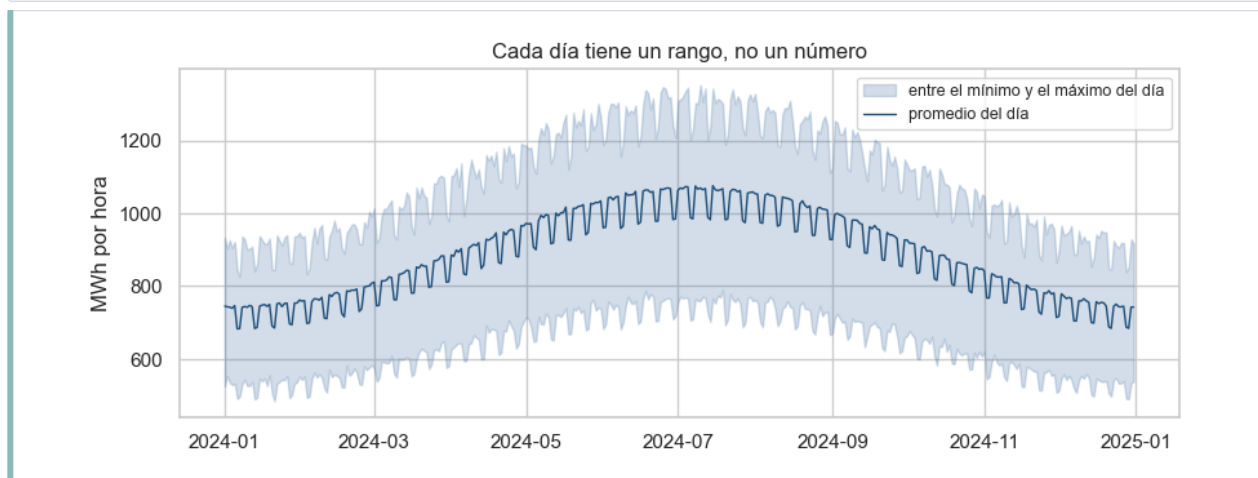


El crudo en transparente y la tendencia encima, en el mismo gráfico. El center=True centra la ventana, sin él la tendencia queda corrida hacia adelante y parece atrasada.

4.4 La banda

```
# Una banda entre el mínimo y el máximo del día dice más que el promedio solo.
minimo = met_h["mwh"].resample("D").min()
maximo = met_h["mwh"].resample("D").max()

fig, ax = plt.subplots(figsize=(10, 3.5))
ax.fill_between(por_dia.index, minimo, maximo, alpha=0.25, color="#4C78A8",
               label="entre el mínimo y el máximo del día")
ax.plot(por_dia.index, por_dia.values, linewidth=1, color="#1F4E79", label="promedio del día")
ax.set_title("Cada día tiene un rango, no un número")
ax.set_ylabel("MWh por hora")
ax.legend(fontsize=9)
plt.show()
```

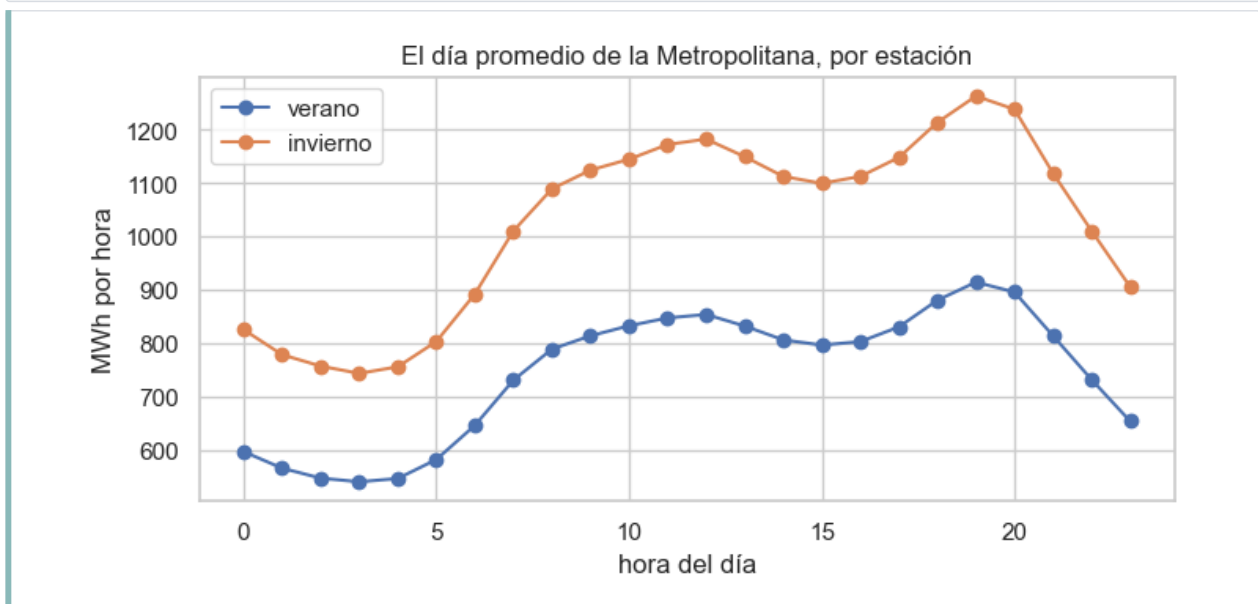


El mínimo y el máximo de cada día como una banda, con el promedio encima. Agrega el rango del día, que es justo lo que importa para dimensionar.

4.5 Verano contra invierno

```
# Y el perfil del día promedio, verano contra invierno.
met_h["mes"] = met_h.index.month
verano = met_h[met_h["mes"].isin([12, 1, 2])].groupby("hora")["mwh"].mean()
invierno = met_h[met_h["mes"].isin([6, 7, 8])].groupby("hora")["mwh"].mean()

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(verano.index, verano.values, marker="o", label="verano")
ax.plot(invierno.index, invierno.values, marker="o", label="invierno")
ax.set_title("El día promedio de la Metropolitana, por estación")
ax.set_xlabel("hora del día")
ax.set_ylabel("MWh por hora")
ax.legend()
plt.show()
```



Dos líneas, veinticuatro puntos cada una, y una pregunta contestada. Este es el tipo de gráfico que se pega en una lámina sin tocar nada.

La solución del ejercicio

El ejercicio pedía cambiar la frecuencia.

```
semanal = met_h["mwh"].resample("W").mean()

fig, ax = plt.subplots(figsize=(10, 3.5))
ax.plot(semanal.index, semanal.values, linewidth=0.8, alpha=0.5, label="semana a semana")
ax.plot(semanal.index, semanal.rolling(4, center=True).mean(),
        linewidth=2, color="#E45756", label="media móvil de 4 semanas")
ax.set_ylabel("MWh por hora")
ax.legend()
plt.show()
```

La historia no cambia, solo cambia el ruido. La estacionalidad se mantiene porque es una señal más fuerte que la frecuencia de muestreo. Comprobar que la conclusión no depende de la decisión de resumen es buena práctica y casi nadie la hace.

Referencia rápida

Instrucción	Para qué sirve
<code>df.set_index(instante)</code>	Poner el tiempo en el índice, obligatorio antes
<code>serie.resample("D").mean()</code>	Bajar la frecuencia. D día, W semana, ME mes
<code>serie.rolling(14, center=True).mean()</code>	Media móvil centrada
<code>ax.fill_between(x, bajo, alto, alpha=)</code>	La banda entre mínimo y máximo
<code>ax.axvline(fecha) · ax.axvspan</code>	Marcar un evento o un periodo
<code>plt.subplots(2, 1, sharex=True)</code>	Dos series con unidades distintas, sin engañar

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>TypeError</code> al hacer <code>resample</code>	El índice no es de tiempo. Hay que hacer <code>set_index</code> antes.
La tendencia va corrida respecto del dato	Falta <code>center=True</code> en el <code>rolling</code> .
Un día tiene un promedio raro	Le faltan horas. Es el Módulo 3 volviendo, revisa los nulos antes de resamplear.

Tres preguntas de repaso

1. Cuáles son las tres formas de resumir una serie larga

Bajar la frecuencia con `resample`, suavizar con una media móvil, y mostrar el rango con una banda entre mínimo y máximo.

2. Qué pierde cada una

El promedio diario pierde la punta de la hora. La media móvil pierde el evento de un día. La banda no pierde el rango, a costa de más tinta.

3. Cómo se elige la ventana de la media móvil

Con criterio del dominio. Siete días saca el efecto del fin de semana, treinta el del mes. Lo que no se hace es ajustarla hasta que aparezca la tendencia que uno quería.

6. Bloque 5, distribuciones

Qué se vio

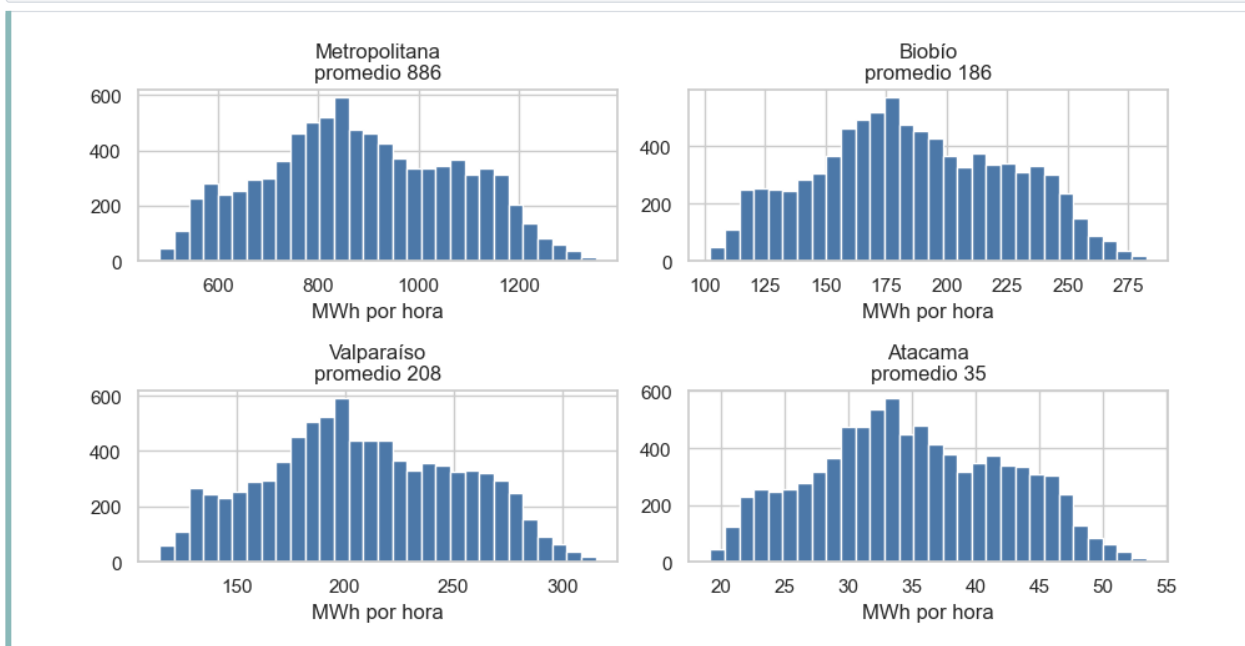
El promedio es el resumen que más informes ha arruinado. Dos cosas con promedios parecidos pueden tener formas completamente distintas, y el promedio no lo dice. Y comparar desviaciones sin mirar el promedio engaña igual.

El código, paso a paso

5.1 Cuatro histogramas

```
# El promedio solo esconde la forma. Cuatro regiones, cuatro histogramas.
cuatro = ["Metropolitana", "Biobío", "Valparaíso", "Atacama"]

fig, ejes = plt.subplots(2, 2, figsize=(9, 5))
for eje, region in zip(ejes.ravel(), cuatro):
    serie = demanda[demanda["region"] == region]["mwh"]
    eje.hist(serie, bins=30, color="#4C78A8")
    eje.set_title(f"{region}\npromedio {serie.mean():.0f}")
    eje.set_xlabel("MWh por hora")
fig.tight_layout()
plt.show()
```

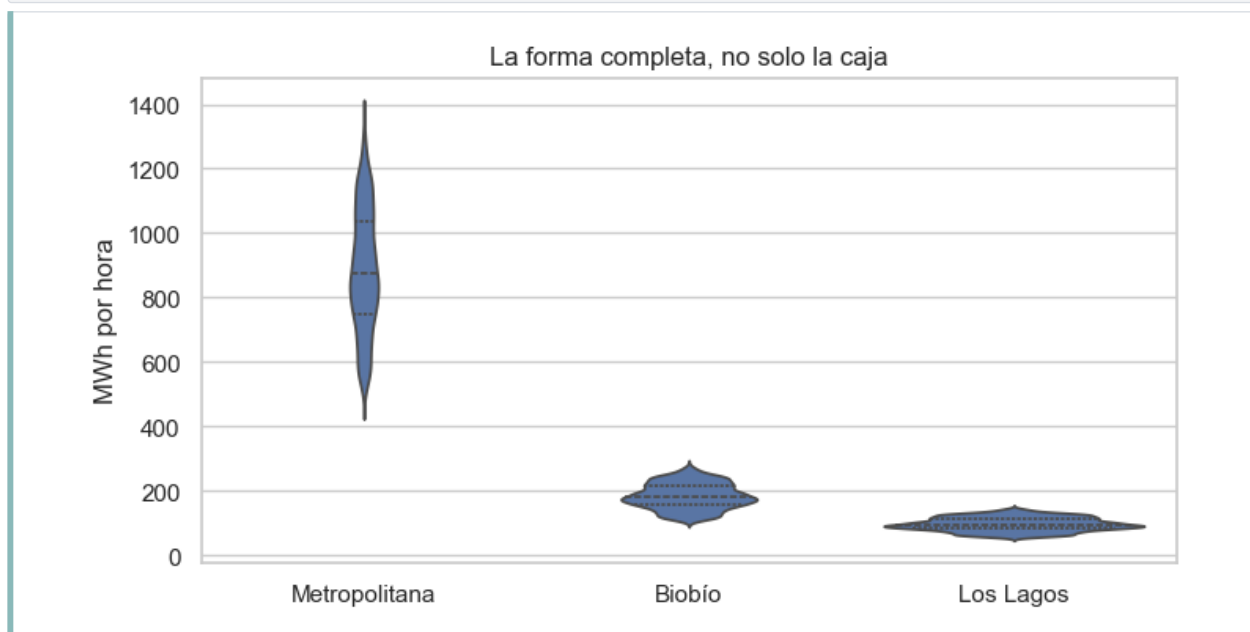


Cuatro regiones con su promedio en el título. Las formas son distintas y los promedios no lo cuentan. La escala horizontal es distinta en cada uno, que es lo que corresponde acá porque las magnitudes no son comparables.

5.2 El violín

```
# El violín muestra la forma y la caja al mismo tiempo.
tres = demanda[demanda["region"].isin(["Metropolitana", "Biobío", "Los Lagos"])]

fig, ax = plt.subplots(figsize=(8, 4))
sns.violinplot(data=tres, x="region", y="mwh", ax=ax, inner="quartile")
ax.set_title("La forma completa, no solo la caja")
ax.set_ylabel("MWh por hora")
ax.set_xlabel("")
plt.show()
```



Muestra la forma completa y adentro las líneas de los cuartiles. Un boxplot escondería una distribución con dos jorobas, el violín no. El ancho a cada altura es cuántas horas hay en ese nivel, o sea un histograma dado vuelta y espejado.

5.3 Los números

```
# La trampa clásica. El promedio no dice nada sobre la variabilidad.
diario = gen.groupby(["tecnologia", "fecha"])["mwh"].sum().unstack(0)
resumen = pd.DataFrame({"promedio": diario.mean().round(1), "desviacion": diario.std().round(1)})
resumen["cv_pct"] = (100 * resumen["desviacion"] / resumen["promedio"]).round(1)

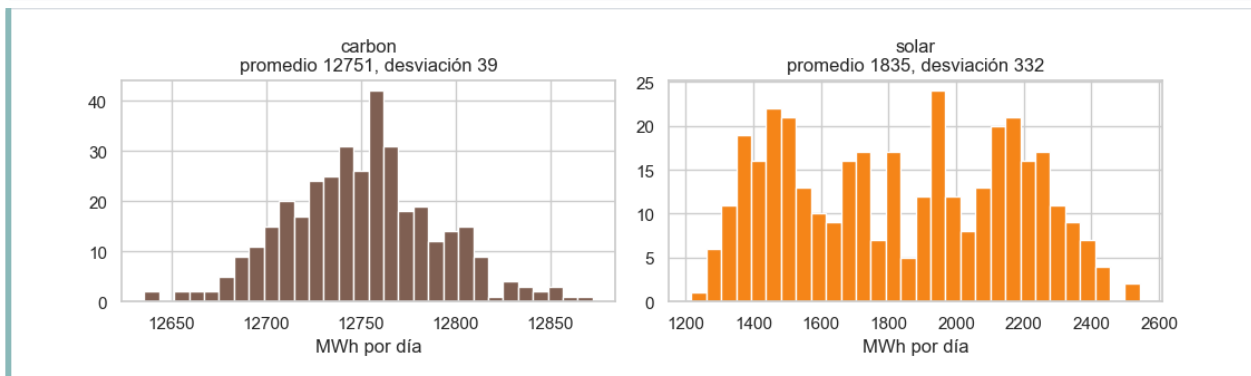
print(resumen.sort_values("cv_pct", ascending=False).to_string())
```

	promedio	desviacion	cv_pct
tecnologia			
solar	1835.2	331.9	18.1
hidro	16684.0	2963.5	17.8
eolica	4278.4	219.0	5.1
diesel	111.2	3.1	2.8
gas	13918.4	72.9	0.5
carbon	12750.9	39.2	0.3

Las seis tecnologías con su promedio, su desviación y el cociente entre las dos. Esa tercera columna es la que ordena de verdad, porque una desviación de 332 sobre un promedio de 1.835 no es lo mismo que una de 2.963 sobre 16.684.

5.4 Y el gráfico

```
# El carbón entrega siete veces más energía que la solar y varía sesenta veces
# menos. Con la escala de cada uno, para que se vea la forma y no el tamaño.
fig, ejes = plt.subplots(1, 2, figsize=(10, 3.2))
for eje, tec, color in [(ejes[0], "carbon", "#7F5F52"), (ejes[1], "solar", "#F58518")]:
    serie = diario[tec]
    eje.hist(serie, bins=30, color=color)
    eje.set_title(f"{tec}\npromedio {serie.mean():.0f}, desviación {serie.std():.0f}")
    eje.set_xlabel("MWh por día")
fig.tight_layout()
plt.show()
```



Carbón y solar, cada uno con su escala. El carbón es una raya y la solar tiene forma. El carbón es más fácil de programar para un despachador, porque se sabe casi exactamente cuánto va a llegar. Con la solar hay que dejar reserva, y eso cuesta. Esa conclusión sale del gráfico, no del promedio.

OJO CON LA EÓLICA DE ESTOS DATOS

En la tabla, la eólica sale como la tecnología más estable del parque, con un coeficiente de variación de 5,1 por ciento contra 18,1 de la solar. En el sistema real es al revés.

Es un artefacto de cómo se generaron estos datos, con ruido independiente hora a hora que al sumar las veinticuatro horas de cuatro parques se promedia y desaparece. En el viento real las rachas duran días, así que la variabilidad no se cancela.

La solución del ejercicio

El ejercicio pedía el factor de planta de cada central.

```
anual = gen.groupby(["central", "potencia_mw"])["mwh"].sum().reset_index()
anual["factor_planta"] = anual["mwh"] / (anual["potencia_mw"] * 8784)

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.hist(anual["factor_planta"], bins=15, color="#4C78A8")
ax.set_title("Factor de planta anual de las veinte centrales, 2024")
ax.set_xlabel("factor de planta")
ax.set_ylabel("cuántas centrales")
plt.show()
```

No sale una sola nube, salen grupos, porque cada tecnología tiene su rango. Es el mismo hallazgo del Módulo 5 visto desde el histograma.

Referencia rápida

Instrucción	Para qué sirve
<code>ax.hist(serie, bins=, alpha=)</code>	Histograma, y con <code>alpha</code> se superponen dos
<code>sns.violinplot(..., inner="quartile")</code>	Forma completa más cuartiles
<code>sns.kdeplot</code>	Densidad suavizada. Cuidado, puede inventar
<code>sns.stripplot</code>	Los puntos mismos, si son pocos

Tres preguntas de repaso

1. Por qué el promedio no basta

Porque no dice nada de la variabilidad. El carbón entrega 12.751 MWh diarios con desviación 39 y la solar 1.835 con desviación 332, y lo que decide casi siempre es el ancho, no el centro.

2. Cuándo conviene el violín en vez del boxplot

Cuando se sospecha que la distribución tiene dos jorobas, porque el boxplot las esconde.

3. Qué trampa tiene la curva de densidad

El suavizado puede inventar densidad donde no hay datos, por ejemplo valores negativos en una variable que no puede serlo.

7. Bloque 6, gráficos interactivos

Qué se vio

Un gráfico interactivo no responde una pregunta, ofrece que el otro la busque. A veces eso es exactamente lo que corresponde, y a veces es delegar el trabajo hacia arriba.

ESTÁTICO	INTERACTIVO
Responde una pregunta	Sirve para explorar
Va a un informe o a una lámina	Va a un archivo que se manda
Se imprime y sigue sirviendo	Impreso pierde todo
matplotlib y seaborn	Plotly
Tú decides qué se mira	El otro decide qué mira

La regla corta. Si el gráfico responde una pregunta, estático. Si sirve para que otro busque la suya, interactivo. El error habitual es entregar un tablero interactivo cuando lo que hacía falta era una figura con una conclusión, y el que lo recibe no sabe qué mirar.

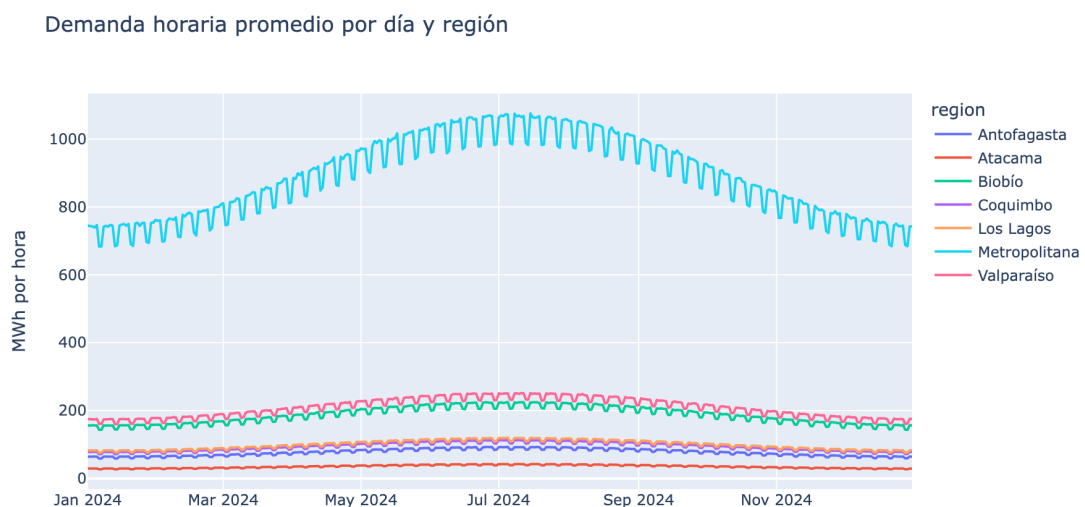
El código, paso a paso

6.1 Plotly express

```
import plotly.express as px

# Plotly arma el gráfico desde el DataFrame, sin fig ni ax.
diaria_region = (demanda.groupby(["fecha", "region"])["mwh"].mean().reset_index())

fig = px.line(diaria_region, x="fecha", y="mwh", color="region",
              title="Demanda horaria promedio por día y región",
              labels={"mwh": "MWh por hora", "fecha": ""})
fig.show()
```



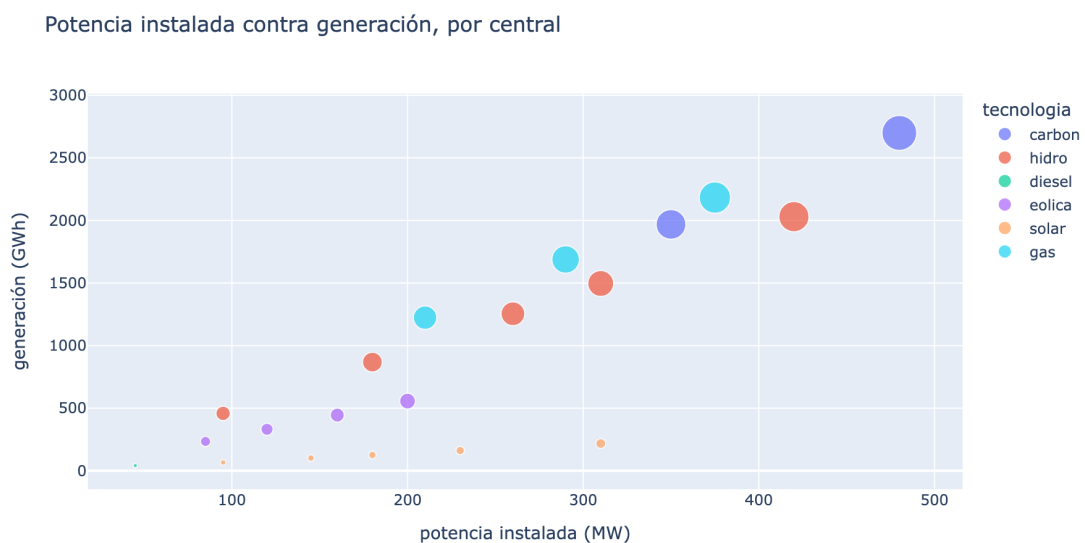
Se le pasa el DataFrame y los nombres de las columnas, y él arma todo. No hay fig ni ax. Acá

el interactivo gana, porque siete líneas superpuestas en estático son un enredo y con un clic en la leyenda se apaga la Metropolitana.

6.2 Con más dimensiones

```
# Con una dimensión más, el color y el tamaño cuentan otra cosa.
resumen = gen.groupby(["central", "tecnologia", "potencia_mw"])[ "mwh"].sum().reset_index()
resumen["gwh"] = (resumen["mwh"] / 1000).round(1)

fig = px.scatter(resumen, x="potencia_mw", y="gwh", color="tecnologia",
                 size="gwh", hover_name="central",
                 title="Potencia instalada contra generación, por central",
                 labels={"potencia_mw": "potencia instalada (MW)", "gwh": "generación (GWh)"})
fig.show()
```



Color por tecnología, tamaño por generación y el nombre al pasar el mouse. El `hover_name` resuelve el problema de los nombres largos, que como etiquetas no caben.

6.3 Guardar el archivo

```
# Un gráfico interactivo se guarda como un archivo que se abre en cualquier navegador.
# Con include_plotlyjs="cdn" la biblioteca no va adentro, el navegador la baja al abrirlo.
# Así el archivo pesa unos KB y no varios MB, a cambio de necesitar internet.
fig.write_html("centrales_interactivo.html", include_plotlyjs="cdn")

import os
print("archivo escrito,", round(os.path.getsize("centrales_interactivo.html") / 1024), "KB")
```

archivo escrito, 12 KB

Un archivo HTML que se manda por correo y se abre en cualquier navegador, sin instalar nada. Con `include_plotlyjs="cdn"` la biblioteca de gráficos no va adentro y el navegador la baja al abrirlo, así que pesa unos kilobytes en vez de varios megabytes, a cambio de necesitar internet.

6.4 Bajarlo a tu computador

```
# El archivo quedó en la máquina temporal de Colab, no en tu computador.  
# Si no lo descargas, se pierde al cerrar la sesión. Esta celda lo baja.  
import sys  
if "google.colab" in sys.modules:  
    from google.colab import files  
    files.download("centrales_interactivo.html")
```

En Colab el archivo queda en la máquina temporal donde corre el notebook, no en tu computador, y se pierde al cerrar la sesión. `files.download` lo baja con el navegador. El `if` hace que la celda no haga nada fuera de Colab, donde el archivo ya queda en la carpeta del notebook.

La solución del ejercicio

El ejercicio pedía barras apiladas por mes.

```
gen["mes"] = gen["fecha"].dt.month  
mensual = (gen.groupby(["mes", "tecnologia"])["mwh"].sum() / 1000).round(1).reset_index()  
  
fig = px.bar(mensual, x="mes", y="mwh", color="tecnologia", barmode="stack",  
             title="Generación mensual por tecnología, 2024",  
             labels={"mwh": "GWh", "mes": "mes"})  
fig.show()
```

El orden de las tecnologías lo decide plotly y se puede fijar con `category_orders`. Recuerda que las barras apiladas solo permiten comparar bien la de abajo, porque es la única con una línea de base común.

CUÁNDO EL INTERACTIVO APORTA Y CUÁNDO ESTORBA

Aporta cuando hay muchas series y quien mira quiere aislar una. Cuando el nombre de cada punto importa y no cabe en la etiqueta. Y cuando el archivo se va a mandar para que otro lo explore.

Estorba cuando el gráfico va a terminar impreso o en una presentación, porque ahí se pierde todo lo interactivo y queda un gráfico peor maquetado. También cuando hay un solo mensaje que dar, porque la interacción invita a buscar otro.

La regla corta. Si el gráfico responde una pregunta, estático. Si sirve para que otro explore, interactivo.

Y un cuidado que importa en una institución pública. El archivo HTML lleva los datos adentro, así que mandarlo es mandar los datos, no solo la imagen.

Referencia rápida

Instrucción	Para qué sirve
<code>px.line(df, x=, y=, color=)</code>	Líneas, una por categoría
<code>px.bar(df, ..., barmode="stack")</code>	Barras apiladas o agrupadas
<code>px.scatter(df, ..., size=, hover_name=)</code>	Dispersión con tamaño y nombre al pasar
<code>labels={col: texto}</code>	Rotular sin renombrar la columna
<code>category_orders={col: lista}</code>	Fijar el orden de las categorías
<code>fig.write_html(archivo)</code>	Un archivo que se abre en cualquier navegador

Tres preguntas de repaso

1. Cuándo conviene un gráfico interactivo

Cuando hay muchas series y hay que aislar una, cuando los nombres no caben, y cuando se manda para que otro explore.

2. Por qué no siempre

Porque impreso pierde todo lo interactivo, y porque si hay un solo mensaje que dar, la interacción invita a buscar otro.

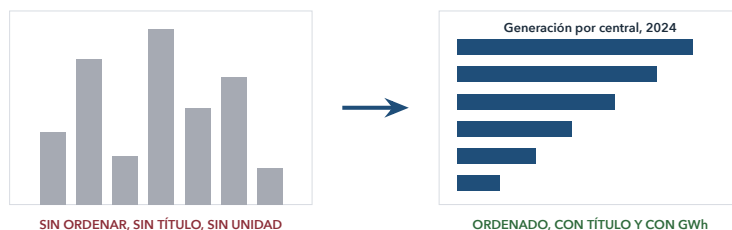
3. Qué cuidado hay con el archivo HTML

Lleva los datos adentro, así que mandarlo equivale a mandar los datos.

8. Bloque 7, antes y después

Qué se vio

Todo el mundo reconoce un gráfico malo cuando lo hizo otro. Casi nadie revisa el propio, porque uno ya sabe lo que quiso decir y lo lee bien aunque esté mal.



Los cuatro arreglos, y ninguno es una técnica. Barras horizontales, porque los nombres largos se leen sin girar la cabeza. Ordenadas, porque sin orden hay que buscar la respuesta y con orden salta sola. Título con el periodo, porque si no, el gráfico no se puede archivar ni citar. Y el valor al lado de la barra cuando son pocas categorías, porque el lector estima mal los largos.

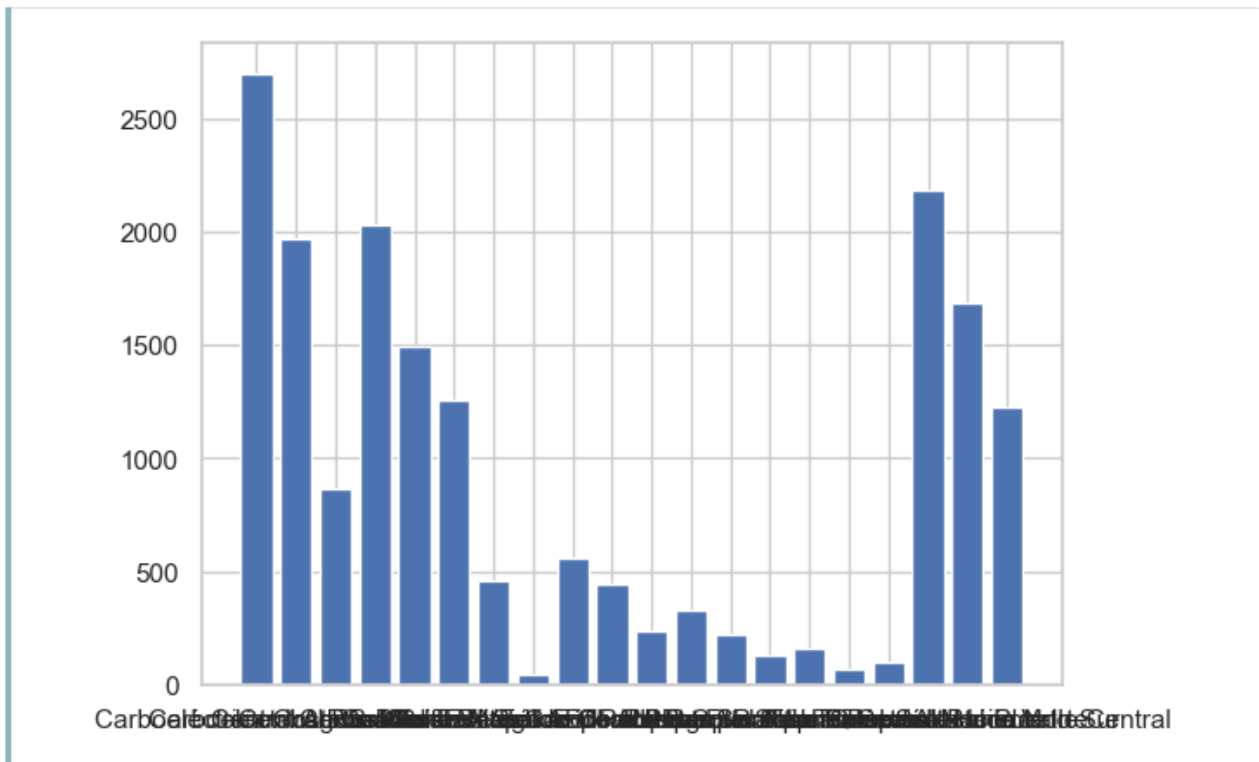
Y el quinto, que es el que más engaña. En un gráfico de barras el eje vertical parte en cero. Cortarlo multiplica visualmente una diferencia que en los números es chica. Hay casos en que se justifica, por ejemplo una serie de temperatura, pero entonces hay que decirlo en el gráfico.

El código, paso a paso

7.1 El gráfico hecho rápido

```
# Un gráfico real, hecho rápido. Todo lo que tiene está mal.
por_central = gen.groupby("central")["mwh"].sum() / 1000

fig, ax = plt.subplots()
ax.bar(por_central.index, por_central.values)
plt.show()
```

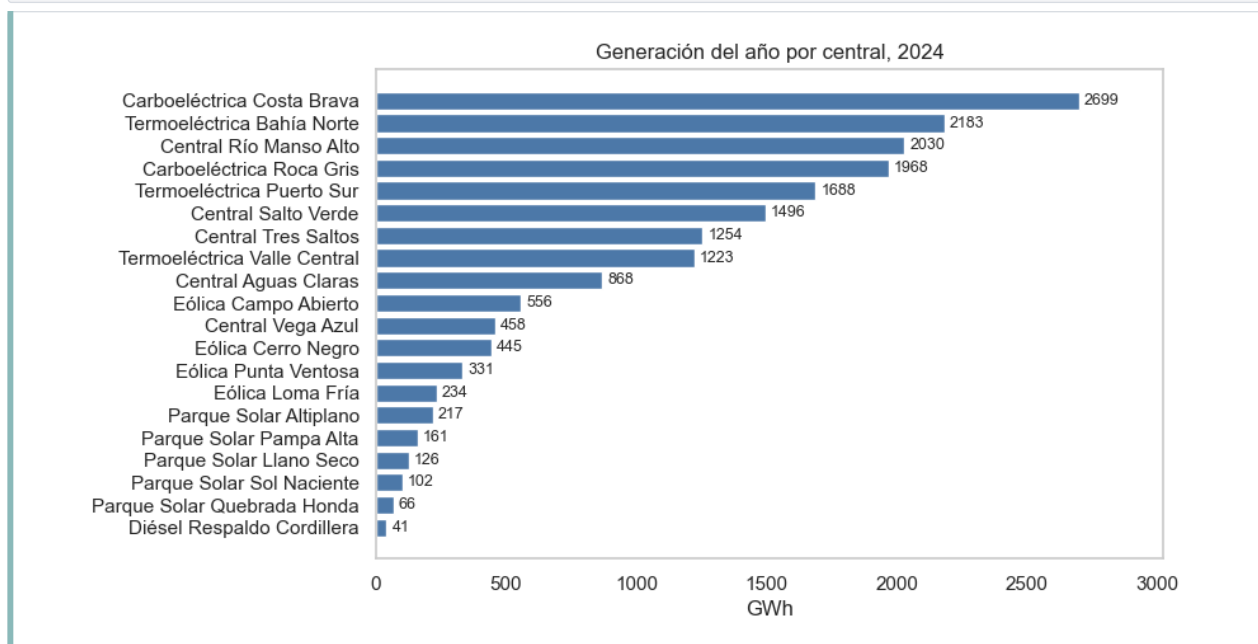


Este es el que sale con apuro. Le falta el título, las unidades, los nombres son ilegibles y, sobre todo, no está ordenado.

7.2 El mismo dato, arreglado

```
# El mismo dato, arreglado. Cuatro cambios, ni uno de ellos es sofisticado.
orden = por_central.sort_values()

fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(orden.index, orden.values, color="#4C78A8")
ax.set_title("Generación del año por central, 2024")
ax.set_xlabel("GWh")
ax.bar_label(ax.containers[0], fmt="%0f", padding=3, fontsize=9)
# Con el valor en cada barra la grilla sobra. El tema de seaborn la deja encendida,
# así que se apaga entera.
ax.grid(False)
ax.set_xlim(0, orden.max() * 1.12)
plt.show()
```

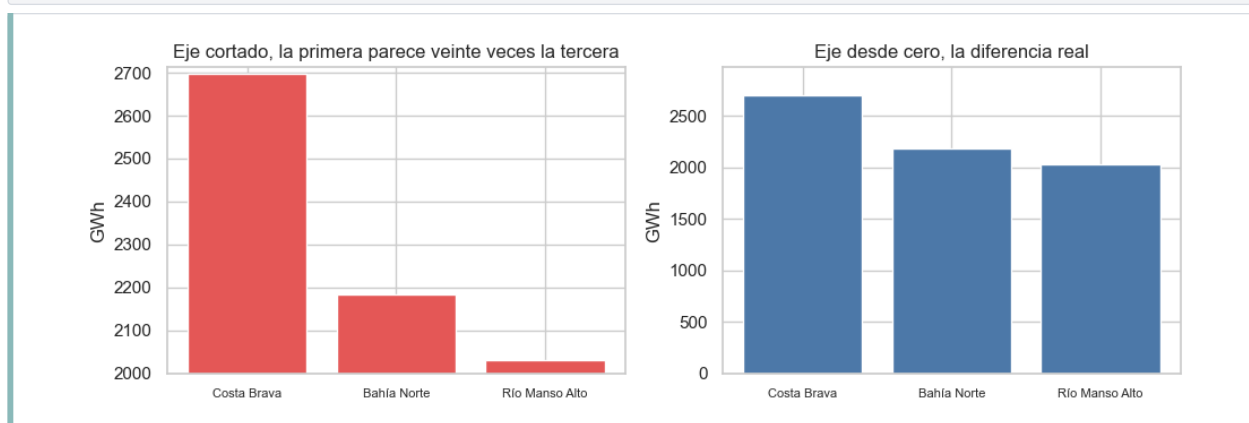


Cuatro cambios y ninguno es una técnica. El `bar_label` es el que menos se conoce y el que más agradece el lector. El `set_xlim` con un doce por ciento extra es para que la etiqueta de la barra más larga no se salga. Y con el valor escrito en cada barra la grilla sobra, así que se apaga con `ax.grid(False)`, entera, porque el tema de seaborn la deja encendida en los dos ejes.

7.3 La escala que engaña

```
# La escala que engaña. El mismo dato, dos historias.
tres_centrales = por_central.sort_values(ascending=False).head(3)
cortos = [c.split(" ", 1)[1] for c in tres_centrales.index] # sin la primera palabra

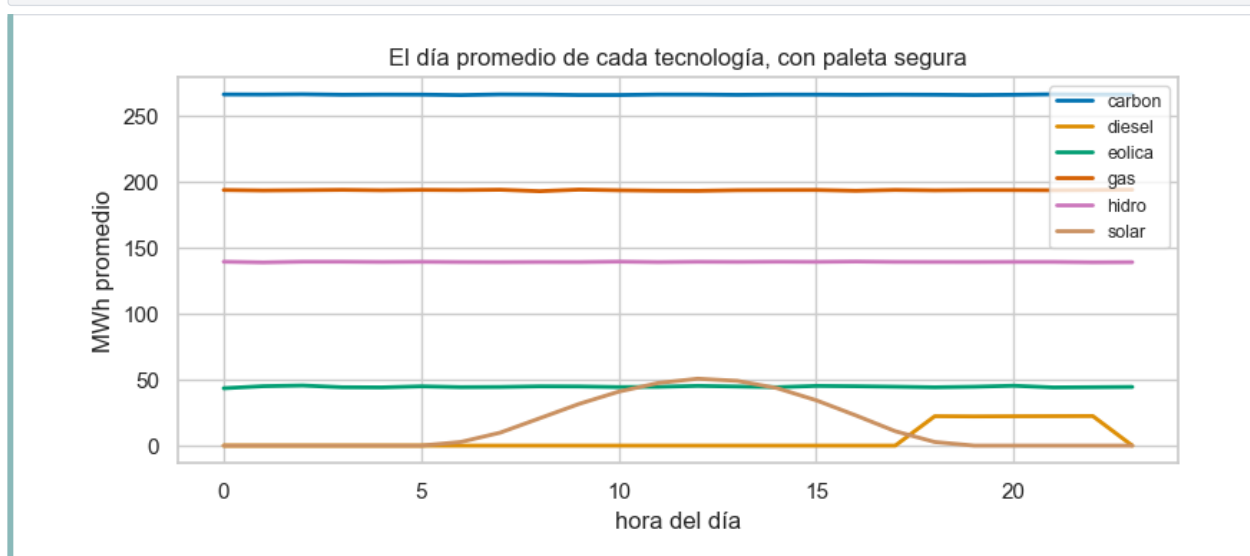
fig, ejes = plt.subplots(1, 2, figsize=(10, 3.5))
ejes[0].bar(range(3), tres_centrales.values, color="#E45756")
ejes[0].set_ylim(tres_centrales.min() * 0.985, tres_centrales.max() * 1.005)
ejes[0].set_title("Eje cortado, la primera parece veinte veces la tercera")
ejes[1].bar(range(3), tres_centrales.values, color="#4C78A8")
ejes[1].set_ylim(0, tres_centrales.max() * 1.1)
ejes[1].set_title("Eje desde cero, la diferencia real")
for eje in ejes:
    eje.set_xticks(range(3))
    eje.set_xticklabels(cortos, fontsize=8)
    eje.set_ylabel("GWh")
fig.tight_layout()
plt.show()
```



El mismo dato, dos veces, las tres centrales que más generan. A la izquierda el eje cortado, donde la primera barra parece veinte veces la tercera. A la derecha desde cero, donde se ve lo que es, un tercio más. El de la izquierda no está mal calculado, está mal hecho. En un gráfico de barras el eje parte en cero, y si no, hay que decirlo en el gráfico.

7.4 La paleta

```
# Y el color. Esta paleta la distingue cualquiera, incluido quien no ve rojo y verde.
fig, ax = plt.subplots(figsize=(9, 3.5))
colores = sns.color_palette("colorblind", n_colors=6)
for (tec, grupo), color in zip(gen.groupby("tecnología"), colores):
    serie = grupo.groupby("hora")["mwh"].mean()
    ax.plot(serie.index, serie.values, label=tec, color=color, linewidth=2)
ax.set_title("El día promedio de cada tecnología, con paleta segura")
ax.set_xlabel("hora del día")
ax.set_ylabel("MWh promedio")
ax.legend(fontsize=9)
plt.show()
```



Esta paleta la distingue cualquiera, incluido quien no ve bien el rojo y el verde, que es alrededor del ocho por ciento de los hombres. Es un requisito, no una cortesía.

LOS CUATRO ARREGLOS

Barras horizontales. Los nombres largos se leen, no hay que girar la cabeza.

Ordenado. Un gráfico de barras sin ordenar obliga a buscar. Ordenado, la respuesta salta.

Título con el periodo. Sin eso, el gráfico no se puede archivar ni citar.

El valor al lado de la barra. Cuando son pocas categorías, el número exacto evita que el lector estime mal.

Y el quinto, el que más engaña. En barras, el eje vertical parte en cero. Cortarlo multiplica visualmente una diferencia que en los números es chica.

Referencia rápida

Instrucción	Para qué sirve
<code>serie.sort_values()</code>	Ordenar, el arreglo que más rinde
<code>ax.bar_label(ax.containers[0], fmt=)</code>	El valor al lado de cada barra
<code>ax.set_xlim(0, max * 1.12)</code>	Espacio para que la etiqueta no se salga
<code>sns.color_palette("colorblind", n)</code>	Paleta que distingue cualquiera

Tres preguntas de repaso

1. Cuáles son los cuatro arreglos

Barras horizontales si los nombres son largos, ordenadas, título con el periodo, y el valor al lado cuando son pocas categorías.

2. Cuándo se puede cortar el eje vertical

En una serie de tiempo sí, porque ahí se lee la variación. En barras no, porque la barra comunica magnitud por su largo y el largo mentiría.

3.Cuál es la prueba más barata para saber si un gráfico engaña

Mostrárselo a alguien que no conozca los datos y preguntarle qué entendió.

Hoja de referencia del Módulo 6

Para imprimir y tener al lado

LA FIGURA

<code>fig, ax = plt.subplots(figsize=)</code>	La hoja y el dibujo
<code>ax.set_title · set_xlabel · set_ylabel</code>	El mínimo, con unidad
<code>fig.tight_layout()</code>	Para que no se pisen
<code>fig.savefig(f, dpi=150, bbox_inches="tight")</code>	Guardar sin borde sobrante

EL TIPO SALE DE LA PREGUNTA

<code>ax.plot</code>	Cómo cambió en el tiempo
<code>ax.barh + sort_values</code>	Cuál categoría es mayor
<code>ax.scatter</code>	Estos dos números se mueven juntos
<code>ax.hist</code>	Cómo se reparte esta variable

COMPARAR Y RESUMIR

<code>sns.boxplot · sns.violinplot</code>	Varios grupos de una vez
<code>sns.heatmap(tabla.div(max, axis=0))</code>	Dos categorías y un número, normalizado
<code>serie.resample("D").mean()</code>	Bajar la frecuencia
<code>serie.rolling(14, center=True).mean()</code>	Suavizar sin bajarla
<code>ax.fill_between(x, min, max)</code>	Mostrar el rango

INTERACTIVO

<code>px.line · px.bar · px.scatter</code>	Desde el DataFrame, sin fig ni ax
<code>fig.write_html(archivo)</code>	Se manda por correo. Lleva los datos adentro

La vuelta de revisión, antes de mandar

- Título con el periodo. Si no, no se puede archivar ni citar.
- Los ejes con su unidad.
- Las barras ordenadas, y horizontales si los nombres son largos.
- El eje desde cero en barras, salvo que puedas justificarlo en el gráfico.
- Paleta que distinga cualquiera. No es cortesía, es un requisito.